SQL fraqmentasiya və defraqmentasiya

Fraqmentasiya məlumat və indeks səhifələrinin iş üçün əlverişsiz yerləşməsi və ya içəridə boş sahənin artması ilə bağlı problemləri təsvir edir. Defraqmentasiya həmin quruluşu daha uyğun formaya gətirmə işidir. Sözlər oxşar olsa da SQL Server indeks fraqmentasiyası, PostgreSQL bloat və əməliyyat sisteminin disk defraqmentasiyası eyni əməliyyat deyil.

İlkin qovluqda bu mövzu üçün müstəqil və əhatəli SQL fraqmentasiya PDF-i tapılmadı. GC materialındakı yaddaş fraqmentasiyası SQL indeks baxımı ilə eyni mövzu deyil. Buna görə bu dərs ayrıca əlavə edildi.

Mündəricat

SQL fraqmentasiya və defraqmentasiya **1**

1 Səhifə və məntiqi sıra . . . 2

2 Xarici daxili və fayl səviyyəli problem . . . 2

3 Page split və yazmalar . . . 2

4 SQL Server-də nəyi ölçürük . . . 3

5 Reorganize nə edir . . . 3

6 Rebuild nə edir . . . 4

7 Hansı halda hansı əməliyyat . . . 4

8 PostgreSQL bloat VACUUM və REINDEX . . . 4

9 Columnstore ayrıca qiymətləndirilir . . . 5

10 Müsahibə cavabı və məşq . . . 5

11 Rəsmi mənbələr . . . 5

1 Səhifə və məntiqi sıra

SQL Server rowstore indeksində məlumat 8 KB-lıq **page**, səhifə bloklarında saxlanır. **B+ tree** axtarış üçün balanslı çoxşaxəli ağacdır. Onun **leaf**, yarpaq, səviyyəsi açar sırasına görə gəzməyə imkan verir. **Logical order** açarların tələb etdiyi sıra, **physical order** səhifələrin fayl daxilindəki yerləşmə sırasıdır.

Məsələn, açar ardıcıllığında səhifələr A, B, C olmalıdır. Faylda yerləşmə A, başqa səhifələr, C, başqa səhifələr, B kimidirsə məntiqi gediş fiziki olaraq ardıcıl olmaya bilər. Burada söhbət cədvəldə yanlış nəticə saxlanmasından getmir. Fraqmentasiya məlumatın korlanması, yəni corruption deyil.

2 Xarici daxili və fayl səviyyəli problem

Logical fragmentation, çox vaxt xarici fraqmentasiya deyilən vəziyyət, indeksin məntiqi sıra ilə fiziki səhifə ardıcıllığının uyğun gəlməməsidir. Geniş aralıq scan-lərində ardıcıl oxuma və read-ahead səmərəliliyinə təsir edə bilər. **Scan** çoxlu səhifəni gəzməkdir. **Read-ahead** baza mühərrikinin lazım olacağını gözlədiyi səhifələri əvvəlcədən gətirməsidir.

Page density səhifənin doluluq nisbətidir. Səhifə daxilində çox boşluq olması daxili fraqmentasiya kimi izah edilir. Eyni məlumat daha çox səhifəyə yayılırsa daha çox oxunuş, RAM və keş sahəsi lazım ola bilər. Boşluq 8 KB səhifənin hər baytının istifadəçi məlumatı olması ilə hesablanmır; səhifənin idarəetmə xərcləri də vardır.

Hesabi nümunə: bir məlumat toplusu yüksək doluluqda 1000 səhifəyə, yarı dolu vəziyyətdə təxminən 2000 səhifəyə yayıla bilər. Dəqiq say sətir ölçüsü və struktur xərclərindən asılıdır. Məqsəd “fraqmentasiya 50 faizdirsə sorğu iki dəfə yavaşdır” demək deyil; eyni məlumat üçün daha çox səhifə daşıma ehtiyacını göstərməkdir.

File system fragmentation isə baza faylının əməliyyat sisteminin saxlama bloklarında yayılmasıdır. SQL `ALTER INDEX` əmri ilə diskin ümumi defraqmentasiyası eyni şey deyil. SSD, yarımkeçirici saxlama qurğusu, ilə HDD, mexaniki disk, təsirləri də eyni deyil. SSD-də fiziki ardıcıllıq problemi zəifləyə bilər, amma aşağı page density-nin oxunuş və yaddaş xərci qalır.

3 Page split və yazmalar

Page split səhifədə yer çatmayanda məlumatın başqa səhifəyə bölüşdürülməsidir. Aralıq açarlarla dolu bir səhifəyə yeni sətir əlavə olunanda yeni səhifə lazım ola bilər. Yeni səhifənin faylın başqa yerində yerləşməsi məntiqi ardıcıllığı poza, iki səhifədə boşluq yarada və əlavə log işi doğura bilər.

`DELETE` boş sahə yarada bilər. Dəyişən uzunluqlu sütunun böyüdüyü `UPDATE` yer çatışmazlığı yarada bilər. Təsadüfi clustered açarlar ağacın müxtəlif yerlərinə insert paylayır və müəyyən iş yüklərində split-ləri artıra bilər. Artan açarlar isə son səhifədə **contention**, yəni eyni resurs uğrunda sıx paralel rəqabət yarada bilər. Ona görə açar seçimi təkcə “təsadüfi pis, artan yaxşı” hökmü ilə həll olunmur.

Fill factor indeks yenidən qurularkən yarpaq səhifələrini hansı hədəf doluluqla yaratmağı bildirir. Məsələn, 90 gələcək aralıq insert-lər üçün təxminən boş yer saxlayır. Bu, hər səhifənin gələcəkdə daim 90 faiz qalması zəmanəti deyil. Aşağı fill factor bəzi split-ləri azaldır, amma başlanğıcdan daha çox səhifə və oxunuş yaradır. Ölçmədən bütün indekslərə aşağı dəyər tətbiq etmək olmaz.

4 SQL Server-də nəyi ölçürük

DMV, Dynamic Management View, serverin iş vəziyyəti barədə məlumat verən sistem görünüşüdür. `sys.dm_db_index_physical_stats` fiziki indeks ölçülərini təqdim edən sistem funksiyasıdır. `avg_fragmentation_in_percent` sıra uyğunsuzluğu, `avg_page_space_used_in_percent` doluluq, `page_count` səhifə sayı üçün istifadə edilir.

T-SQL

```
SELECT
    i.name,
    s.partition_number,
    s.page_count,
    s.avg_fragmentation_in_percent,
    s.avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats
(
    DB_ID(), OBJECT_ID(N'dbo.Orders'),
    NULL, NULL, 'SAMPLED'
) AS s
JOIN sys.indexes AS i
  ON i.object_id = s.object_id
 AND i.index_id = s.index_id
WHERE s.index_level = 0
  AND s.alloc_unit_type_desc = 'IN_ROW_DATA'
  AND i.type IN (1, 2);
```

Bu diaqnostika yalnız `dbo.Orders` üçün nümunədir. `DB_ID()` cari baza ID-sini, `OBJECT_ID` obyekt ID-sini verir. **Partition** cədvəl və ya indeksin bölünmüş hissəsidir; hissələr ayrıca fərqli vəziyyətdə ola bilər. `index_level = 0` yarpaq səviyyəsidir. `IN_ROW_DATA` sətir daxilində saxlanan məlumatın ayırma vahidid ir. `type IN (1,2)` rowstore clustered və nonclustered indeksləri seçir.

`SAMPLED` nümunə götürərək analiz edir; kiçik indeksdə daha tam analizə keçə bilər. `LIMITED` daha yüngül məlumat verir və bəzi doluluq göstəriciləri NULL ola bilər. `DETAILED` daha çox səhifə oxuyur. Bu sorğular da resurs və kilid yükü yarada bilər. Geniş server analizi əvəzinə hədəf indeksdən başlamaq daha ölçülə bilən yanaşmadır. Göstəricilərə çıxış üçün uyğun server və baza icazəsi tələb olunur.

5 Reorganize nə edir

REORGANIZE rowstore indeksinin yarpaq səviyyəsində səhifə sırasını və sıxlığını yaxşılaşdıran daha mərhələli baxımdır. Mövcud indeks üzərində işləyir. Tam yenidənqurmadan az resurs tələb edə bilər, lakin çox böyük problem üçün daha uzun çəkə bilər.

T-SQL

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REORGANIZE;
```

Bu əmr indeksi yeni adla yaratmır. Adətən online xarakterlidir, yəni xidmət davam edə bilər, amma resurs və kilid təsirini sıfırlamır. **Lock** verilənlərə paralel giriş qaydasını qoruyan kiliddir. REORGANIZE indeks statistikasını ayrıca yeniləmir. Statistics dəyərlərin paylanmasını təsvir edir və optimizer-in plan qərarına təsir edir. Lazımdırsa statistikaya ayrıca baxım qərarı verilir.

6 Rebuild nə edir

REBUILD indeksi yenidən qurur. Uyğun səhifə sırası və seçilmiş fill factor ilə yeni quruluş yaranır. CPU, I/O, əlavə disk, transaction log və bəzi ssenarilərdə `tempdb` xərci yüksək ola bilər. I/O disk və şəbəkə giriş çıxışıdır. `tempdb` SQL Server-in aralıq işlər üçün istifadə etdiyi sistem bazasıdır.

T-SQL

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REBUILD;
```

Offline rebuild istifadəçi girişini müəyyən müddət bloklaya bilər. **Online rebuild** uyğun xüsusiyyət olduqda daha çox paralel girişə imkan verir, amma başlanğıc və son mərhələlərdə kilidlər yenə mümkündür. Versiya, edition, indeks tipi və digər məhdudiyyətlər yoxlanmadan `ONLINE = ON` hər yerdə işləyəcək sayılmır. **Edition** məhsulun imkan paketidir.

Uyğun versiya və indekslərdə **resumable rebuild** işi dayandırıb davam etdirməyə imkan verir. Bunun şərtləri ayrıca yoxlanmalıdır. Bütün indeksi rebuild etmək həmin indeks statistikasını da yeniləyir; amma cədvəlin bütün digər sütun statistikalarının hamısını avtomatik yeniləmir. Partition rebuild və başqa xüsusiyyətlərdə statistika təfərrüatı ayrıca nəzərə alınır.

7 Hansı halda hansı əməliyyat

Vəziyyət	Düşünüləcək qərar
Kiçik indeks yüksək faiz göstərir	Səhifə sayı azdırsa praktiki qazanc olmaya bilər. Faizə baxıb avtomatik baxım etmə.
Böyük aralıq scan yavaşdır	Fraqmentasiya, density, storage və oxunuşu birlikdə qiymətləndir.
Tək açarla dar seek normaldır	Yüksək fraqmentasiya təkbaşına təcili rebuild səbəbi deyil.
Səhifələr çox boşdur	Density yaxşılaşmasının oxunuş və yaddaş qazancını qiymətləndir.
Plan yanlış sətir sayı hesablayır	Statistikaya və parametrə bax; indeks quruluşu əsas səbəb olmaya bilər.
Yazma yükü çoxdur	Baxımın log, replika gecikməsi, kilid və əlavə split təsirini hesabla.

Tarixi 5 faiz və 30 faiz kimi hədlər bəzi köhnə şablonlarda görünür. Bunlar bütün sistemlər üçün məcburi qərar qaydası deyil. Microsoft-un aktual yanaşması iş yükü, page density və ölçülən performans qazancına əsaslanmağı vurğulayır. **Workload** sistemin real oxuma və yazma əməliyyatları toplusudur.

İş axını belədir: yavaş sorğunu ölç, indeks ölçüsünü və doluluğu yoxla, səbəbi təsdiqlə, bir indeksdə seçilmiş baxımı apar, sonra eyni sorğunu müqayisə et. **Maintenance window** istifadəçilərə təsirin qəbul olunan aşağı yük vaxtıdır. Disk və log sahəsi, bloklanma, replikasiya gecikməsi və işi dayandırma imkanları nəzərə alınır. Replika başqa serverdə saxlanan məlumat nüsxəsidir.

8 PostgreSQL bloat VACUUM və REINDEX

PostgreSQL **MVCC**, Multi Version Concurrency Control, ilə eyni sətrin müxtəlif versiyalarını saxlayaraq paralel oxuma və yazmanı idarə edir. Artıq heç bir aktiv transaction-a lazım olmayan köhnə sətir versiyası **dead tuple** adlanır. Tuple burada sətir versiyasıdır. **Bloat** cədvəl və ya indeksin faydalı məlumatına nisbətən lazımsız böyüməsidir.

VACUUM təmizlənə bilən köhnə versiyaları idarə edir və sahəni gələcək təkrar istifadə üçün azad edir. Adi VACUUM ümumən faylı sıxıb bütün boşluğu əməliyyat sisteminə qaytarmır; müəyyən son boş səhifələr üçün istisnalar ola bilər. **ANALYZE** planlayıcı statistikalarını yeniləyir. **Autovacuum** bu işləri avtomatik görən fon mexanizmidir.

VACUUM FULL cədvəli yenidən yazaraq sıxlaşdıra və yer qaytara bilər, amma güclü kilid və əlavə sahə tələb edir. **REINDEX** indeksin yeni nüsxəsini qurur. `REINDEX ... CONCURRENTLY` uyğun hallarda yazmaları daha az bloklamağa çalışır, əvəzində əlavə vaxt və iş görür. Məhdudiyyət və versiya qaydaları var.

PostgreSQL

```
VACUUM (ANALYZE) public.orders;

REINDEX INDEX CONCURRENTLY public.ix_orders_customer_date;
```

Bu iki əmr eyni işi görmür. Birincisi cədvəlin versiya təmizliyi və statistikaya, ikincisi indeksin yenidən qurulmasına aiddir. SQL Server `REORGANIZE` əmrinin PostgreSQL-də eyni adlı qarşılığı kimi təqdim edilməməlidir. Uzun transaction-lar köhnə versiyaların təmizlənməsini gecikdirə bilər; təkcə REINDEX kök səbəbi həll etməyə bilər.

9 Columnstore ayrıca qiymətləndirilir

Columnstore məlumatı sütun əsaslı və sıxılmış qruplarda saxlayan indeks ailəsidir. Analitik hesabatlarda çox sətirdən az sütun oxumaq üçün faydalıdır. **Rowgroup** birlikdə saxlanan sətir qrupudur. **Delta store** hələ əsas sıxılmış qrupa keçməmiş məlumatın saxlandığı hissədir. Silinmiş sətir faizi və kiçik rowgroup-lar burada baxım qərarına təsir edir.

Rowstore B+ tree-dəki `avg_fragmentation_in_percent` məntiqini columnstore-a olduğu kimi tətbiq etmək olmaz. Columnstore REORGANIZE və REBUILD semantikası da öz xüsusiyyətlərinə malikdir. Buna görə əvvəl indeksin tipini müəyyən et, sonra uyğun ölçünü seç.

10 Müsahibə cavabı və məşq

Fraqmentasiya niyə yavaşıda bilər? Məntiqi sıra ilə fiziki səhifə sırası pozulduqda geniş oxumalar daha səmərəsiz ola bilər. Səhifə doluluğu aşağı olduqda eyni məlumat üçün daha çox səhifə oxunur və RAM tutulur. Təsirin ölçüsü sorğudan və storage-dən asılıdır.

Rebuild hər yavaş bazanı düzəldirmi? Xeyr. Yanlış indeks, bloklanma, bahalı JOIN, yanlış statistika və ya tətbiqdə N+1 səbəb ola bilər. Baxım yalnız uyğun fiziki quruluş problemini hədəfləyir.

Məşq. Test bazasında bir indeks üçün page_count, density və fragmentation göstəricilərini qeyd et. Aralıq SELECT-in logical read və vaxtını ölç. Reorganize və ya rebuild-dən sonra ölçməni təkrarla. Dəyişiklik azdırsa bu da nəticədir: faiz yaxşılaşması həmişə istifadəçi vaxtının yaxşılaşması demək deyil.

11 Rəsmi mənbələr

- Microsoft Learn - Optimize index maintenance
- PostgreSQL - Routine vacuuming
- PostgreSQL - REINDEX

Yoxlama tarixi 28 oktyabr 2026. Nümunələr diaqnostika və tədris üçündür; real bazada heç bir baxım əmri icra edilməyib.